# Projet 2 — Détecter des attaques réseau avec le Machine Learning (Réseaux / Sécurité / IoT)

**Niveau** : L3 · **Durée conseillée** : 1 à 2 semaines · **Travail** : individuel · **Langage** : Python (pandas, scikit-learn)

## Contexte

Un système de détection d'intrusion (IDS) doit décider, pour chaque **flux réseau**, s'il est **normal (0)** ou une **attaque (1)** : scan de ports, force brute SSH/Telnet, ou déni de service (DoS). Tu vas entraîner un modèle qui apprend à faire cette distinction à partir d'exemples.

## Les données (`data/flux_train.csv`, 3000 flux)

| Colonne | Signification |
|---|---|
| `duration` | durée du flux (s) — quelques valeurs manquantes |
| `protocol` | TCP, UDP ou ICMP |
| `dst_port` | port de destination |
| `src_bytes` / `dst_bytes` | octets envoyés / reçus |
| `nb_packets` | nombre de paquets |
| `nb_failed_logins` | échecs d'authentification |
| `nb_conn_same_host` | connexions récentes vers la même machine |
| `syn_rate` | proportion de paquets SYN (0 à 1) |
| **`attaque`** | **cible** : 0 = normal, 1 = attaque |

Les classes ne sont pas équilibrées (≈ 28 % d'attaques) : **l'accuracy seule ne suffit pas**, utilise le **F1-score** de la classe « attaque ».

## Ce que tu dois faire

Écris **`src/train.py`** (entraînement) et **`src/predict.py`** (prédiction), puis remplis la section « Résultats » du README de ton dépôt.

1. **Explorer** rapidement les données (taille, valeurs manquantes, répartition de `attaque`).
2. **Prétraiter** : imputer les valeurs manquantes, mettre à l'échelle, encoder `protocol` (dans un `Pipeline`).
3. **Entraîner au moins 2 modèles** (ex. régression logistique + arbre de décision ou forêt aléatoire).
4. **Évaluer** sur un jeu de validation : `classification_report`, **matrice de confusion**, **F1 de la classe attaque**.

5. **Choisir le meilleur**, le ré-entraîner sur toutes les données, l'enregistrer dans `models/model.joblib`.
6. **`src/predict.py`** doit contenir la fonction `predict` décrite ci-dessous : elle charge `models/model.joblib` (un `Pipeline` qui inclut le prétraitement) et prédit.
7. **Analyser** : combien d'attaques ratées (faux négatifs) ? Pourquoi est-ce le pire cas en sécurité ?

### Fonction `predict` attendue (ne pas changer le nom ni la signature)

Le correcteur automatique appelle cette fonction :

```
def predict(df: pd.DataFrame) -> list:
    # df : mêmes colonnes que flux_train.csv, SANS 'attaque'
    # retourne une liste de 0 (normal) ou 1 (attaque), une valeur par ligne
```

> 🔒 L'enseignant testera ton modèle sur des flux **jamais vus** (jeu de test caché). Ne triche pas en cherchant la solution : un bon modèle généralise.

## Rendu sur GitHub

1. Crée un dépôt **public** nommé `projet-ml-NOM-Prenom`.
2. Pousse tout le travail (au moins **3 commits** avec des messages clairs).
3. Respecte la structure du dépôt indiquée ci-dessous.
4. Envoie le **lien du dépôt** au responsable de classe avec ton **nom et prénom**.

### Structure attendue du dépôt

```
projet-ml-NOM-Prenom/
├── README.md           (copie la section « Résultats » ci-dessous et remplis-la)
├── requirements.txt    (avec les versions des bibliothèques)
├── data/flux_train.csv
├── src/train.py
├── src/predict.py
└── models/model.joblib
```

Pour obtenir les versions : `pip freeze | grep -i -E "pandas|scikit|joblib|numpy"`

## Barème (/20)

- **12 points automatiques** : structure du dépôt (3), `predict` fonctionne (3), performance F1 sur le test caché (6).
- **8 points qualité** : comparaison d'au moins 2 modèles avec métriques dans le README (3), analyse des erreurs / matrice de confusion (2), qualité du code (2), historique de commits (1).

| F1 sur test caché | Points (/6) |
|---|---|
| ≥ 0,95 | 6 |
| ≥ 0,90 | 5 |
| ≥ 0,85 | 4 |
| ≥ 0,80 | 3 |
| ≥ 0,70 | 2 |
| > 0,50 | 1 |

---

## Résultats (à remplir par l'étudiant)

**Nom / Prénom :**

| Modèle | F1 (attaque) | Rappel (attaque) | Précision (attaque) |
|---|---|---|---|
| Régression logistique | | | |
| (2e modèle) | | | |

**Modèle retenu et pourquoi :**

**Analyse des erreurs (faux négatifs / faux positifs) :**